INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Práctica 1: Regular Expressions

NATURAL LANGUAGE PROCESSING

7CM2

INTEGRANTES:

VÁZQUEZ BLANCAS CESAR SAID

PROFESOR:

JORGE LUIS ROSAS TRIGUEROS

FECHA DE ENTREGA: 9 DE MARZO DE 2026

FECHA DE REALIZACION DE LA PRACTICA: 7 DE MARZO DE 2026

Marco Teórico

Las expresiones regulares (Regular Expressions o Regex) son secuencias de caracteres que definen patrones de búsqueda dentro de cadenas de texto. Estas herramientas son ampliamente utilizadas en programación y procesamiento de datos para identificar, extraer o validar información dentro de textos. Según la documentación oficial de Python, las expresiones regulares permiten describir conjuntos de cadenas mediante patrones específicos que pueden ser utilizados para realizar búsquedas complejas dentro de textos (Python Software Foundation, 2024).

Las expresiones regulares están formadas por caracteres literales y metacaracteres. Los caracteres literales representan símbolos exactos que deben aparecer en el texto, mientras que los metacaracteres poseen un significado especial que permite construir patrones más flexibles. Por ejemplo, el símbolo . representa cualquier carácter, mientras que operadores como *, + o {} permiten indicar repeticiones de un patrón determinado. Gracias a estas características, las expresiones regulares son una herramienta poderosa para el análisis y manipulación de texto (Friedl, 2006).

En el lenguaje de programación Python, las expresiones regulares se implementan mediante el módulo re. Este módulo proporciona funciones especializadas que permiten buscar coincidencias entre un patrón y una cadena de texto. Algunas de las funciones más utilizadas son re.search(), re.match(), re.findall() y re.sub(). Estas funciones permiten localizar coincidencias dentro de cadenas, obtener todas las ocurrencias de un patrón o incluso reemplazar partes del texto que coincidan con una expresión regular (Python Software Foundation, 2024).

Cuando una expresión regular encuentra una coincidencia dentro de una cadena, el módulo re devuelve un objeto denominado match. Este objeto contiene información importante sobre la coincidencia encontrada, como la posición en la que inicia y termina dentro del texto. A partir de estos datos es posible extraer o manipular la parte específica del texto que coincide con el patrón definido.

Las expresiones regulares tienen aplicaciones en diversas áreas de la informática, como la validación de datos, el análisis de registros de sistemas (logs), la extracción de información en grandes volúmenes de texto y el procesamiento de lenguaje natural. Por ejemplo, pueden utilizarse para verificar si una cadena corresponde al formato de un correo electrónico, detectar números telefónicos dentro de un documento o identificar patrones específicos dentro de archivos de registro.

En la presente práctica de laboratorio se emplean expresiones regulares en Python para analizar listas de cadenas de texto. El objetivo es identificar las coincidencias con un patrón determinado y mostrar visualmente la parte de la cadena que coincide con dicho patrón.

Para mejorar la visualización de los resultados, se utilizan códigos ANSI de color, los cuales permiten resaltar la coincidencia dentro de la salida mostrada en la terminal.

El desarrollo de prácticas de laboratorio como esta permite comprender de manera aplicada el funcionamiento de las expresiones regulares y su implementación dentro de un lenguaje de programación. Además, fomenta el desarrollo de habilidades relacionadas con el procesamiento de texto y la automatización de tareas, que son fundamentales en diversas áreas de la computación.

Material y Equipo

Para la realización de esta práctica de laboratorio se utilizaron los siguientes recursos de hardware y software:

Hardware

- Computadora personal o laptop con capacidad para ejecutar un entorno de programación.
- Procesador compatible con sistemas operativos modernos.
- Memoria RAM suficiente para ejecutar herramientas de desarrollo.

Software

- **Sistema operativo**: Windows / Linux / macOS.
- **Python 3.x**: lenguaje de programación utilizado para desarrollar el programa de la práctica.
- **Módulo re de Python**: biblioteca utilizada para trabajar con expresiones regulares.
- **Editor de código o entorno de desarrollo (IDE),** por ejemplo:
    - Visual Studio Code
    - PyCharm
    - IDLE de Python
- **Terminal o consola del sistema,** utilizada para ejecutar el programa y visualizar los resultados.

Archivos utilizados

- Script de Python desarrollado para analizar cadenas de texto mediante expresiones regulares.
- Listas de cadenas de prueba utilizadas para verificar el funcionamiento del patrón definido en la práctica.

Desarrollo de la Practica

Definición del problema

El objetivo de la práctica fue utilizar expresiones regulares en Python para analizar cadenas de texto y detectar patrones específicos dentro de ellas. Para lograr esto, se desarrolló un programa que recibe listas de palabras y una expresión regular, con el fin de identificar coincidencias dentro de las cadenas.

El programa debe mostrar las palabras que coinciden con el patrón definido y resaltar visualmente la parte de la cadena que coincide con la expresión regular.

Definición de las listas de prueba

En primer lugar, se definieron dos listas de cadenas de texto. La primera lista contiene las palabras en las que se espera encontrar coincidencias con la expresión regular. La segunda lista contiene palabras que no deben coincidir con el patrón.

```
Match = ['wazzzzzup', 'wazzzup']
Skip = ['wazup', 'wazzup' ]
```

Figura 1: Definición de listas

La lista Match contiene palabras en las cuales se buscará el patrón definido por la expresión regular. Por otro lado, la lista Skip contiene palabras que no presentan coincidencias con el patrón.

Definición de la expresión regular

Posteriormente se definió la expresión regular que se utilizará para realizar la búsqueda dentro de las cadenas de texto.

```
myRegEx = r"..zzz.."
```

Figura 2: Definición de Regex

Esta expresión regular está compuesta por los siguientes elementos:

- . representa cualquier carácter.
- zzz representa una secuencia literal de tres letras "z".
- . representa nuevamente cualquier carácter.

Por lo tanto, el patrón busca cualquier carácter seguido de tres letras "z" y posteriormente cualquier carácter adicional.

Importación del módulo de expresiones regulares

Para poder trabajar con expresiones regulares en Python es necesario utilizar el módulo **re**, el cual forma parte de la biblioteca estándar del lenguaje.

```
import re
```

Figura 3: Import de re, librería para regex en Python

Este módulo proporciona diversas funciones que permiten buscar coincidencias entre un patrón y una cadena de texto.

Definición de colores para resaltar coincidencias

Para facilitar la visualización de las coincidencias encontradas, se utilizaron códigos ANSI para mostrar texto en color dentro de la terminal.

```
GREEN = "\033[92m"
RESET = "\033[0m"
```

Figura 4: Definición de colores a usar

El código \033[92m permite mostrar el texto en color verde, mientras que \033[om restablece el formato normal del texto.

Función para resaltar coincidencias

Posteriormente se implementó una función que recibe una cadena de texto y una expresión regular. Esta función busca coincidencias dentro de la cadena y resalta la parte correspondiente utilizando los códigos de color definidos anteriormente.

```python
def highlight_match(text, regex):
    match = re.search(regex, text)
    if match:
        start, end = match.span()
        highlighted = (
            text[:start] +
            GREEN + text[start:end] + RESET +
            text[end:]
        )
        return highlighted
    return text
```

Figura 5: Función principal de la practica

La función utiliza re.search() para encontrar la primera coincidencia del patrón dentro de la cadena. Si se encuentra una coincidencia, se obtiene la posición inicial y final mediante el método span(). Posteriormente, la cadena se reconstruye insertando los códigos de color alrededor de la parte que coincide con la expresión regular.

Ejecución del programa

Finalmente, se recorren las listas definidas anteriormente para aplicar la función de búsqueda y mostrar los resultados en la terminal.

```python
print("Match:")
for word in Match:
    highlighted = highlight_match(word, myRegEx)
    print(f"{word} : {highlighted}")

print("\nSkip:")
for word in Skip:
    highlighted = highlight_match(word, myRegEx)
    print(f"{word}:{highlighted}")
```

Figura 6: Recorrido de listas para ver resultados

Durante esta etapa el programa analiza cada palabra de la lista Match y muestra la coincidencia resaltada en color. Posteriormente se muestran las palabras de la lista Skip, las cuales no presentan coincidencias con el patrón definido.

Resultados obtenidos

Al ejecutar el programa se observó que las palabras de la lista Match contienen coincidencias con el patrón definido por la expresión regular, mientras que la palabra incluida en la lista Skip no presenta coincidencias.

Esto permite comprobar que el programa identifica correctamente los patrones dentro de las cadenas y resalta visualmente las coincidencias encontradas.

```
Match:
wazzzzzup : wazzzzzup
wazzzup : wazzzup

Skip:
wazup:wazup
wazzup:wazzup
```

Figura 7: Resultados de la ejecución del codigo

Conclusiones y recomendaciones

Durante el desarrollo de esta práctica se logró comprender el funcionamiento de las expresiones regulares y su implementación dentro del lenguaje de programación Python. A través del uso del módulo re, fue posible identificar patrones específicos dentro de cadenas de texto, lo cual demuestra la utilidad de las expresiones regulares para el procesamiento y análisis de información textual.

Uno de los aspectos más importantes de la práctica fue aprender cómo las expresiones regulares permiten definir patrones de búsqueda utilizando metacaracteres. Esto facilita la detección de secuencias específicas dentro de cadenas de texto sin necesidad de realizar múltiples comparaciones manuales.

Asimismo, se observó que el uso de herramientas como los códigos ANSI para colorear la salida en la terminal permite visualizar de manera más clara las coincidencias encontradas por el programa. Esto resulta útil para verificar el funcionamiento correcto del patrón definido en la expresión regular.

Otra observación relevante es que las expresiones regulares son herramientas muy utilizadas en diversas áreas de la informática, como la validación de datos, el análisis de registros de sistemas y el procesamiento de grandes volúmenes de texto. Por esta razón, su comprensión y aplicación práctica resulta fundamental para el desarrollo de software.

Como recomendación, sería útil realizar más ejercicios que involucren distintos tipos de patrones y metacaracteres para fortalecer la comprensión del funcionamiento de las

expresiones regulares. De igual forma, se recomienda explorar otras funciones del módulo re de Python para ampliar las posibilidades de análisis y manipulación de texto.

Durante el desarrollo de la práctica se realizaron algunos experimentos adicionales con el programa con el objetivo de comprender mejor el funcionamiento de las expresiones regulares.

Uno de los experimentos consistió en modificar la expresión regular utilizada en el programa para observar cómo cambiaban los resultados obtenidos. Por ejemplo, se probaron diferentes patrones como z+, el cual permite detectar una o más repeticiones del carácter "z". Al aplicar este patrón a las cadenas de prueba, se observó que el programa detectaba cualquier secuencia de una o más letras "z" dentro de las palabras analizadas.

Otro experimento realizado fue agregar nuevas palabras a las listas de prueba para analizar cómo respondía el programa ante diferentes casos. Por ejemplo, se añadieron palabras como wazzzzup, wazzzzzzzup y wazzzp. Esto permitió verificar que el programa es capaz de identificar correctamente las coincidencias con el patrón definido y resaltar la parte correspondiente del texto.

También se realizaron pruebas modificando la expresión regular para buscar diferentes combinaciones de caracteres, lo cual permitió comprender cómo los metacaracteres influyen en la detección de patrones dentro de las cadenas de texto.

A partir de estos experimentos se pudo observar que las expresiones regulares son una herramienta muy flexible y poderosa para el análisis de texto, ya que permiten definir patrones complejos de manera sencilla. Además, estas pruebas ayudaron a reforzar la comprensión del funcionamiento del módulo re de Python y de las técnicas utilizadas para resaltar coincidencias dentro de la salida del programa.

En general, la experimentación adicional permitió comprender mejor cómo funcionan las expresiones regulares y cómo pueden adaptarse a distintos problemas relacionados con el procesamiento de texto.

Bibliografia

Python Software Foundation.
Regular Expression Operations (re module).
Python Documentation, 2024.
Disponible en: https://docs.python.org/3/library/re.html

Friedl, Jeffrey.
Mastering Regular Expressions.
O'Reilly Media, 2006.